



AUBURN

UNIVERSITY

Virtual Memory

Overview

Objectives

- Review the **objectives, constraints, and “helpers”** of Memory Management.
- Learn and understand the **design guiding principles** to hierarchical memory.
- Describe, discuss and evaluate **the virtual memory (Demand Paging)**.
- Learn and understand **Page Replacement** Policies.
- Learn and Understand **Frame Allocation** Policies.

Requirements

- Basics from Computer Organization principles (COMP 3350 or equivalent)
- Read Sections 1.8.3, 8.1 through 8.6.
- Section 8.3 is not covered by this presentation



Memory Management Objectives, Constraints, and “Helpers”

- **Objectives**

1. Offer what a user/programmer wants

a flat, **fast**, **inexpensive**, and **infinite** space starting at **Address 0**

2. Maximize the number of processes in the memory
3. Minimize the access time to memory

- **Constraints**

Faster memory costs much more

- **“Helpers”**

1. Only a small part of a program (code, data, and stack) is needed in memory

2. Temporal locality (referenced recently will be referenced again)

3. Spatial locality (referenced things close will be referenced)

- (re) Read Section 1.8.3

Objectives

- ? The user/programmer wants **fast, inexpensive, unbounded, convenient** memory space;
 - ? **convenience** of a flat and simple space is offered using the **logical** space (OS and memory management unit take care of the messy details and realities of the **physical** space)
 - ? **fast** : it would be easy if the user gave up inexpensive
 - ? **inexpensive**: it would be easy if the user can live with a slow system
 - ? **unbounded**: easy if the user gave up inexpensive
 - ? What to do?

- ? The OS must keep the CPU busy ? must keep as many ready processes as possible in the system.... (**need large memory**)

Constraints : Faster Costs More

? Static RAM (SRAM)

? 0.5ns – 2.5ns, \$2000 – \$5000 per GB

? Dynamic RAM (DRAM)

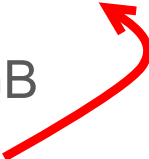
? 50ns – 70ns, \$20 – \$75 per GB

? Flash

? 25µs – 50µs, \$0.40 – \$1 per GB

? Magnetic disk

? 5ms – 20ms, \$0.20 – \$2 per GB



50 to 100 more expensive

Helpers

- ? Only a small part of a program (code, data, and stack) is needed in memory
 - ? At the extreme, we need only space for one instruction and its operands
- ? Most (well designed) programs exhibit temporal and spatial locality:
 - ? **Temporal** locality: a referenced item will be soon referenced again
 - ? **Spatial** locality : neighbors of a recently referenced item are often also referenced right after

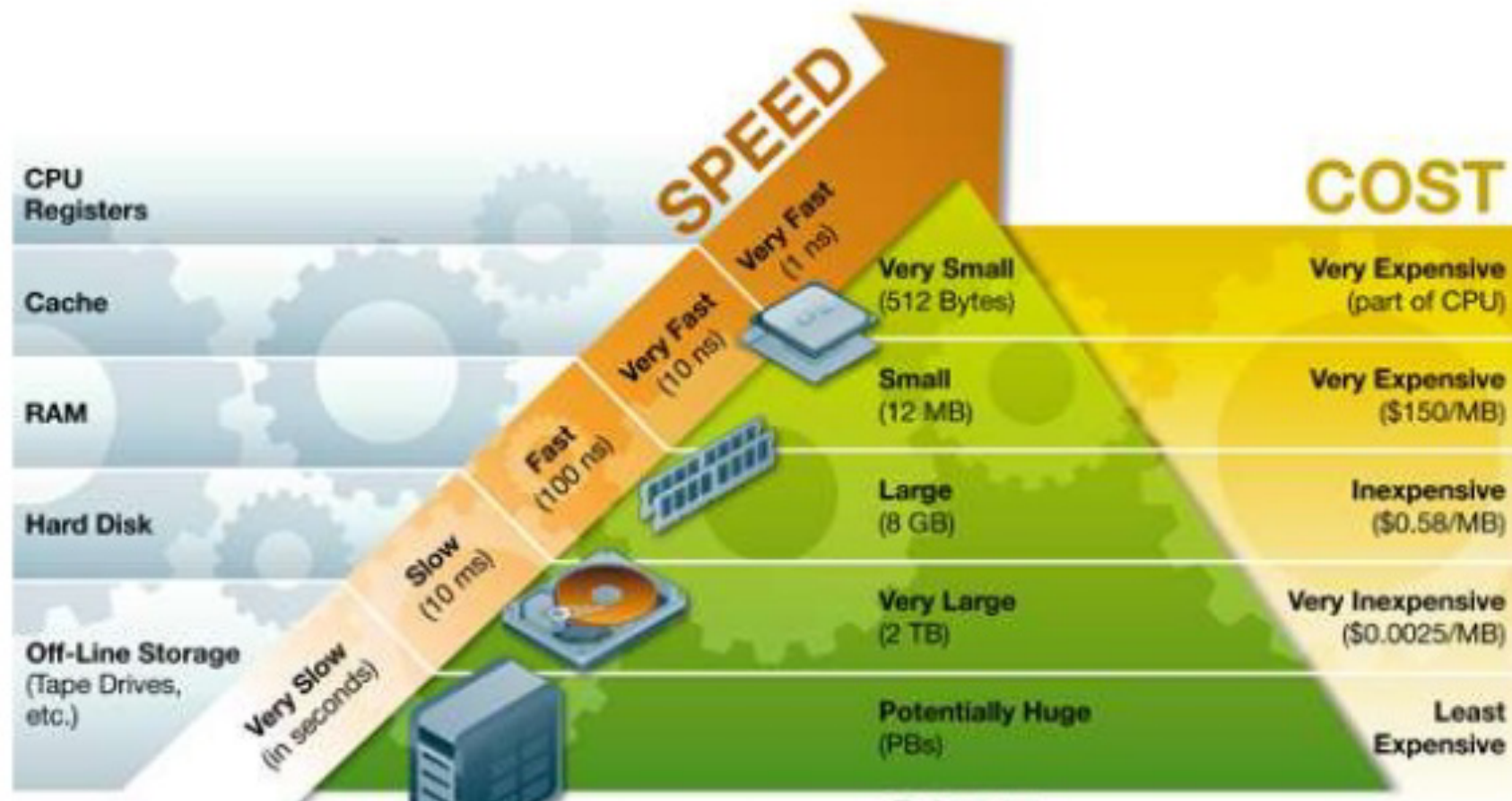
Conclusion : keep in the memory only a small part of a program, especially that part often used! If small.... costs less...
Then the memory could be **PHYSICALLY**

Hierarchical Memory

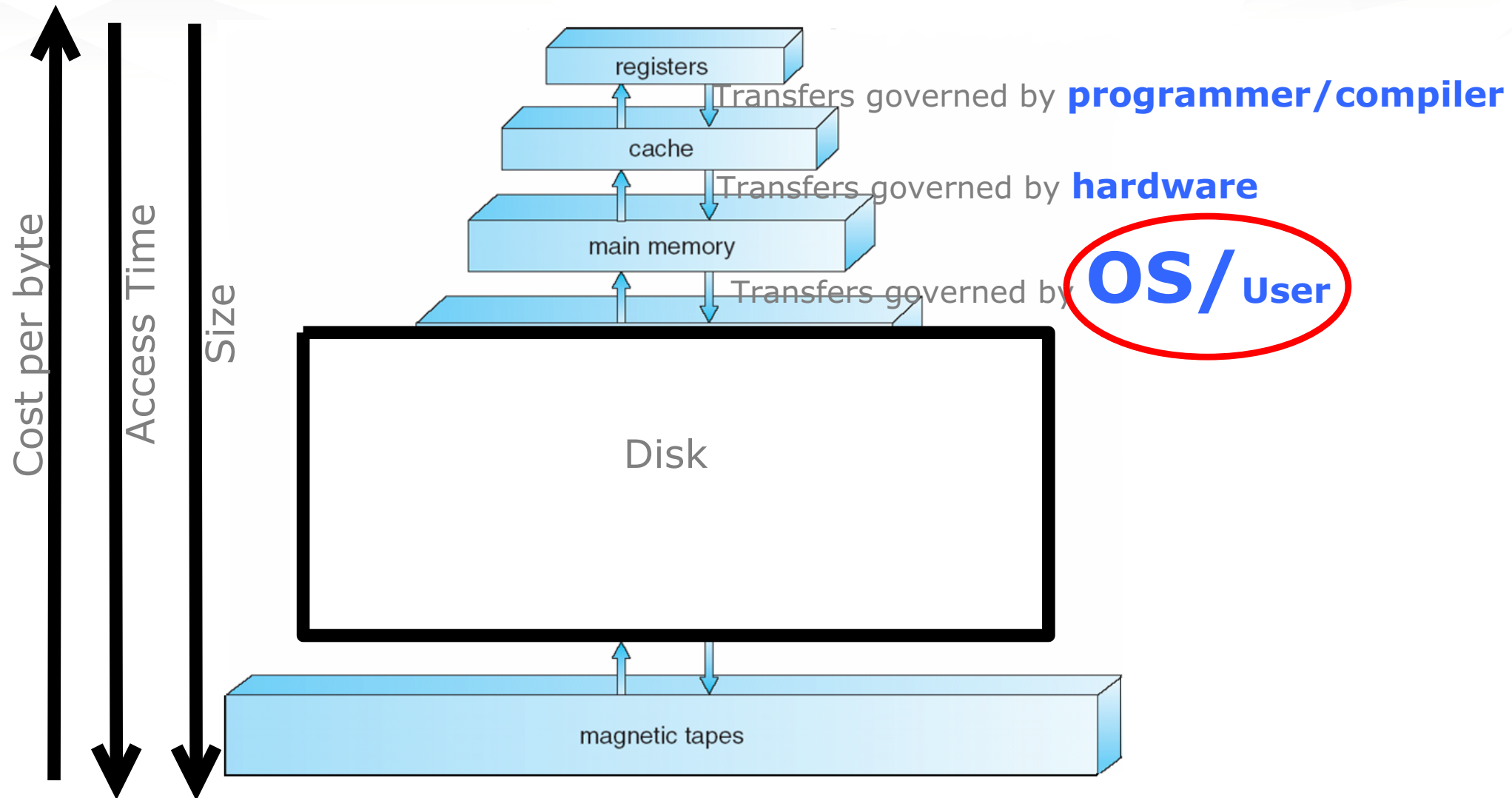
- Build a memory such that:
 - the **most used** part of a program remains in a **fast** storage **closest** to the CPU
 - The closest storage does not have to be large because the CPU does not need the full program at once
- **(re) Read Section 1.8.3**

Hierarchical Memory

Extended Memory Hierarchy



Memory System Hierarchy



Performance of Various Levels of Storage

☐ Movement between levels of storage hierarchy can be explicit or implicit

Price	?	?	?	?
Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

Objective of Virtual Memory

? **ILLUSION**: (achieved by the OS helped by the MMU)

? Make the user/programmer believe that the physical space is flat and homogeneous (while it is **hierarchical** and implemented with **heteroclite** technologies)

? Separation of user logical space from physical memory

? Very large logical (virtual) memory

? Efficient use of physical memory consisting of

1. the main memory and
2. the secondary storage secondary

? Implementation

? **Demand paging**: only bring in a page when it is demanded (data or instruction referenced)

Demand Paging

- How does Demand Paging work?
 - What happens when a page is not in the main memory?
 - **Answer:** a **page fault** occurs?
 - How to handle a page fault?
-
- Read Sections 8.2 Introduction and 8.2.1

On Demand Paging

0	A
1	B
2	C
3	D
4	E
5	F

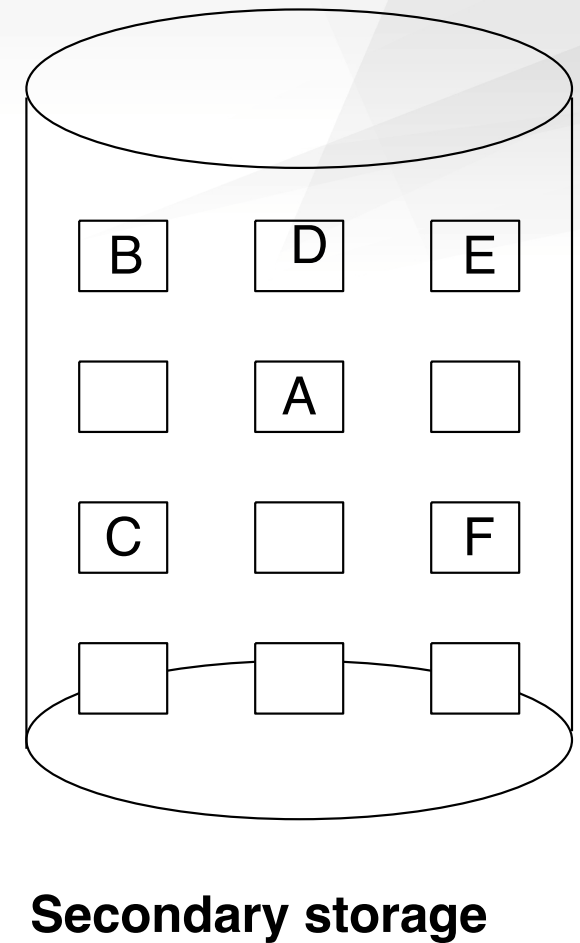
Logical Space

0	1
1	
2	3
3	
4	
5	0

Page Table

0	F
1	A
2	
3	C

Main Memory



- What happens when we refer **Page # 2**?
- If the main memory was always sufficiently large to accommodate the logical space then we would stop here....
- What happens when we refer **Page # 1**?

On Demand Paging

0	A
1	B
2	C
3	D
4	E
5	F

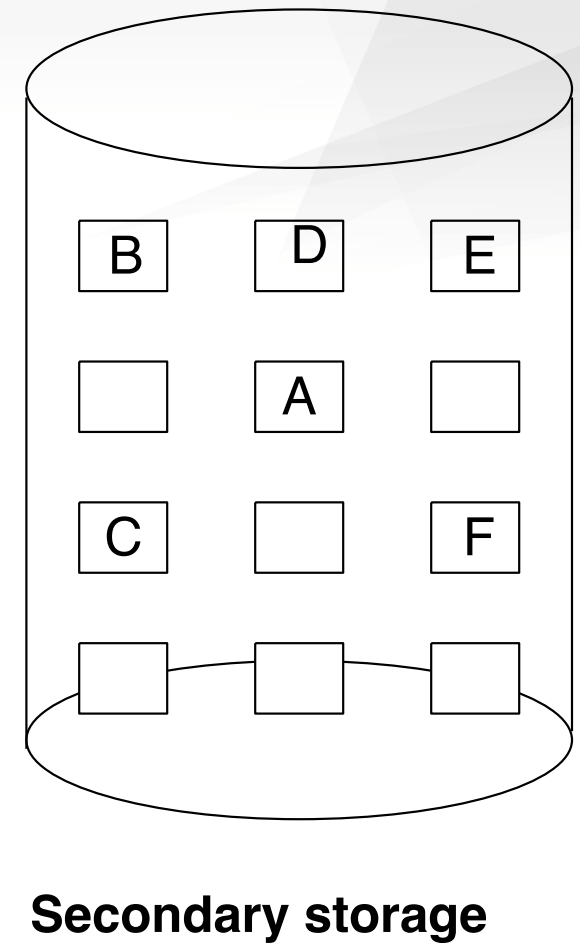
Logical Space

0	1	v
1		i
2	3	v
3		i
4		i
5	0	v

Page Table

0	F
1	A
2	
3	C

Main Memory



- What happens when we refer **Page # 2**?
- If the main memory was always sufficiently large to accommodate the logical space then we would stop here....
- What happens when we refer **Page # 1**?

Handling a Memory reference (with potential **Page Fault**)

CPU generates a memory reference for data or instruction

Hardware

1. Check the process' page table to see if the page is in main memory.
2. If it is in the main memory, access it as usual
3. If it is not, a **page fault** occurs, interrupt to OS (trap)
4.
 - a) Suspend (not block) the (offending) process
 - b) Locate a free frame in memory (how?)
 - c) Locate the missing page from disk.
5. Bring in the missing page into memory (meanwhile, other processes may use the CPU, **How/Why/Who?**)
6. Update page table
7.
 - a) Reset the process's PC so that we restart with the offending instruction.
 - b) Change the status of process from "suspended" to "ready"

Some Issues...

? Is this time efficient?

- Here we go again, the system checks first in the main memory and may have to check in the secondary storage...

? How do we find a free frame in the main memory when none is left?

- We may have to swap out another page? How to select it (**victim**)?

? How to restart a offending instruction that already modified other addresses?

? Some solutions:

- ? Determine if there would be a page fault before any modification occurs

- ? Hold results temporarily until it is certain there is no page fault. Then, proceed with modification.

Time Performance of Demand Paging (Effective Access Time)

? What is the effective (average) access time taken by a memory access ?

? Let us derive it:

? **Ma** : (average) time for a memory access without page fault (e.g. $ma = 100 \text{ ns}$)

? **Page fault penalty time**: (average) time taken by a page fault:

? O.S. service time when a page fault is detected $100 \text{ } \mu\text{s}$

? Time by disk to read the page into memory 25 milliseconds

? Time to restart the process (around $100 \text{ } \mu\text{s}$)

? We can approximate the total of page fault time as about $25,000,000 \text{ ns}$

Let p be the page fault rate: probability that a page fault occurs when a memory reference is made

Effective Access Time = $(1-p) \cdot ma + p \cdot \text{page fault penalty time}$

$$= ma - p \cdot ma + p \cdot \text{page fault time}$$

$$= 0.100 - 0.1p + 25,000 p \text{ (} \mu\text{s)}$$

$$\approx 0.100 + 25,000 p \text{ (} \mu\text{s)}$$

Time Performance of Demand Paging (Effective Access Time)

? Effective Access Time $\approx 0.100 + 25,000 p$ (?s)

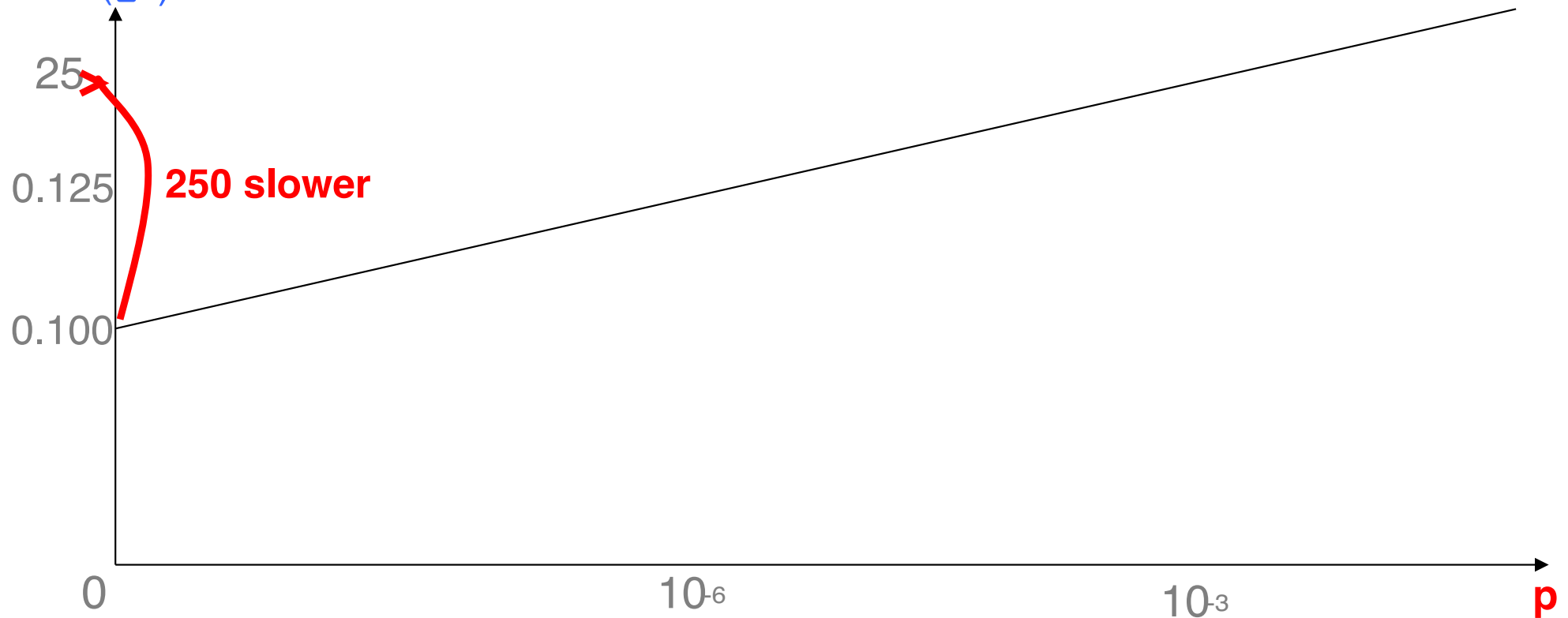
? As **software** OS designers: what is our power to reduce the effective access time?

? The 0.100 constant is the access time to the main memory (Can we do anything?)

? The 25,000 comes from the access time to the disk.... (Can we do anything?)

? **Answer**: the only parameter in our reach is: **the page fault rate p** .

e.a.t (?s)



Policies Affecting The Page Fault Rate p

? Main Memory Management:

- ? Memory allocation: How many frames to allocate to each process?
- ? How do we manage the process pool of frames when a page fault occurs? In other words, how do we choose a page to swap out when a page fault occurs and we must bring in a page?
 - ? This is called **page replacement**.

Page Replacement Policies

- What is page replacement?
- Page replacement policies:
 - First In First Out (FIFO)
 - Optimal
 - Least Recently Used (LRU)
 - LRU-approximation implementation
- [Read Section 8.4.1-8.4.3](#)

Page Replacement

? Page replacement is the process to select a page to replace in order to free a frame.

? After a page fault, the OS will locate a free frame as follows:

? if a frame is available, then pick it

? otherwise, find a page to swap out: page replacement consists of selecting a ‘victim’.

? Consequences:

? If the victim is referenced just after being replaced, this makes the page fault worse...

? Replacement policies:

? **FIFO**: First In First Out

? **OPTIMAL**

? **LRU**: Least Recently Used:

FIFO Page Replacement Policy

- FIFO (First In First Out)
 - replace the page which came earliest

Pages Referenced		7	0	1	2	0	3	0	4	2	3																																	
Memory State	<table><tr><td>-</td></tr><tr><td>-</td></tr><tr><td>-</td></tr></table>	-	-	-	<table><tr><td>7</td></tr><tr><td>-</td></tr><tr><td>-</td></tr></table>	7	-	-	<table><tr><td>7</td></tr><tr><td>0</td></tr><tr><td>-</td></tr></table>	7	0	-	<table><tr><td>7</td></tr><tr><td>0</td></tr><tr><td>1</td></tr></table>	7	0	1	<table><tr><td>2</td></tr><tr><td>0</td></tr><tr><td>1</td></tr></table>	2	0	1	<table><tr><td>2</td></tr><tr><td>0</td></tr><tr><td>1</td></tr></table>	2	0	1	<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>1</td></tr></table>	2	3	1	<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>0</td></tr></table>	2	3	0	<table><tr><td>4</td></tr><tr><td>3</td></tr><tr><td>0</td></tr></table>	4	3	0	<table><tr><td>4</td></tr><tr><td>2</td></tr><tr><td>0</td></tr></table>	4	2	0	<table><tr><td>4</td></tr><tr><td>2</td></tr><tr><td>3</td></tr></table>	4	2	3
-																																												
-																																												
-																																												
7																																												
-																																												
-																																												
7																																												
0																																												
-																																												
7																																												
0																																												
1																																												
2																																												
0																																												
1																																												
2																																												
0																																												
1																																												
2																																												
3																																												
1																																												
2																																												
3																																												
0																																												
4																																												
3																																												
0																																												
4																																												
2																																												
0																																												
4																																												
2																																												
3																																												
Page Fault ?		y	y	y	y	n	y	y	y	y	y																																	

Page Fault Rate = Number of page faults/Number of references = 9/10 = 0.9

Optimal Page Replacement Algorithm

? FIFO does not work well:

? FIFO does not consider the pattern of the reference. FIFO is blind to the pattern access..

? The Optimal algorithm (OPT)

? Replace the page that will not be used for the longest time (psychic !!!)

? OPT looks when the page is to be used.

OPT vs FIFO

pfr= 6/10

OPT

7	7	7	2	2	2	2	2	2	2
-	0	0	0	0	0	0	4	4	4
-	-	1	1	1	3	3	3	3	3
y	y	y	y	n	y	n	y	n	n
7	0	1	2	0	3	0	4	2	3
7	7	7	2	2	2	2	4	4	4
-	0	0	0	0	3	3	3	2	2
-	-	1	1	1	1	0	0	0	3
y	y	y	y	n	y	y	y	y	y

FIFO

pfr= 9/10

1. Optimal (OPT) delivers the minimal page fault rate
2. But, OPT cannot be implemented as **future** references cannot be predicted with certainty
3. What to do?
4. Use the fact that programs, in general, exhibits a good temporal locality
5. Recall: temporal locality means that an item just referenced will be referenced soon.
6. This rule translates into: replace the **Least Recently Used (LRU)** page in the **past**.

Least Recently Used

- What is LRU?
 - LRU approximates OPTIMAL policy
 - Implementation of strict LRU is cumbersome...
 - Look at approximate implementation of LRU
-
- [Read Sections 8.4.4-8.4.5](#)

Least Recently Used (LRU)

? OPT needs the knowledge of future references

? Reasonable Assumption:

? If good temporal locality then a page recently used is likely to be used in the near future.

? A page unused recently is unlikely to be used in the near future.

? This assumption leads to a good **solution approximation**

? The **Least Recently Used Algorithm (LRU)**:

? Replace the page that has not been used for the longest period.

OPT versus LRU

pfr= 6/10

OPT

7	7	7	2	2	2	2	2	2	2
-	0	0	0	0	0	0	4	4	4
-	-	1	1	1	3	3	3	3	3
y	y	y	y	n	y	n	y	n	n
7	0	1	2	0	3	0	4	2	3
7	7	7	2	2	2	2	4	4	4
-	0	0	0	0	0	0	0	0	3
-	-	1	1	1	3	3	3	2	2
y	y	y	y	n	y	n	y	y	y

LRU

pfr= 8/10

1. Optimal (OPT) delivers the minimal page fault rate
2. But, OPT cannot be implemented as **future** references cannot be predicted with certainty
3. What to do?
4. Use the fact that programs, in general, exhibits a good temporal locality
5. Recall: temporal locality means that an item just referenced will be referenced soon.
6. This rule translates into: replace the **Least Recently Used (LRU)** page in the **past**.

Implementations of LRU

1) Implementation I: **Time-of-use** register in every entry of **page table**.

Page i

Frame base address	Time-of-use

1) Implementation II: **Stack** of page numbers

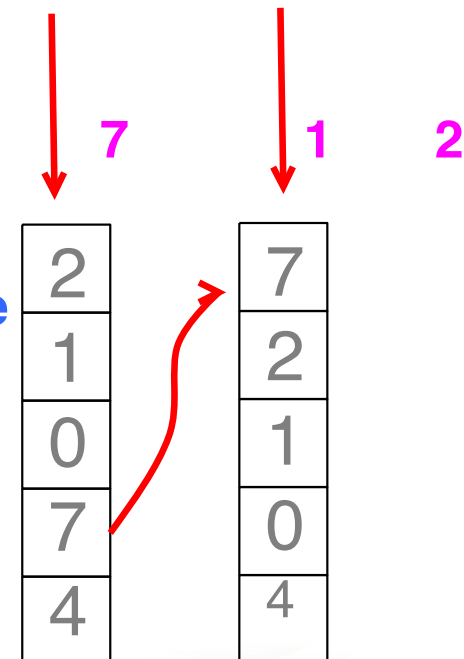
Reference string :

4 7 0 7 1 0 1 2 1 2

These two implementations are not easy in hardware

What to do?

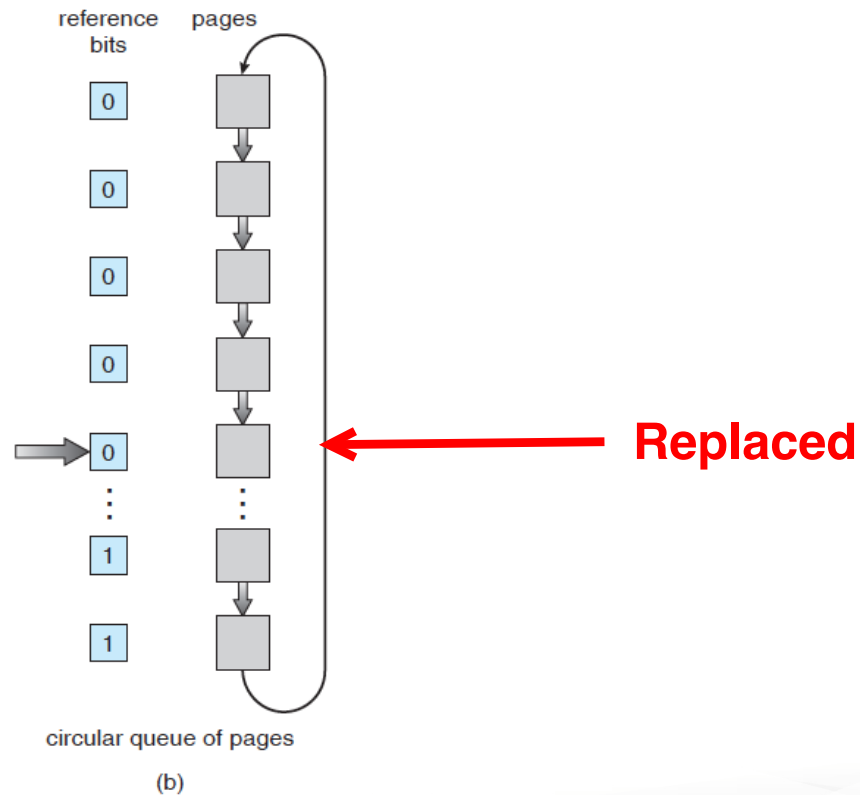
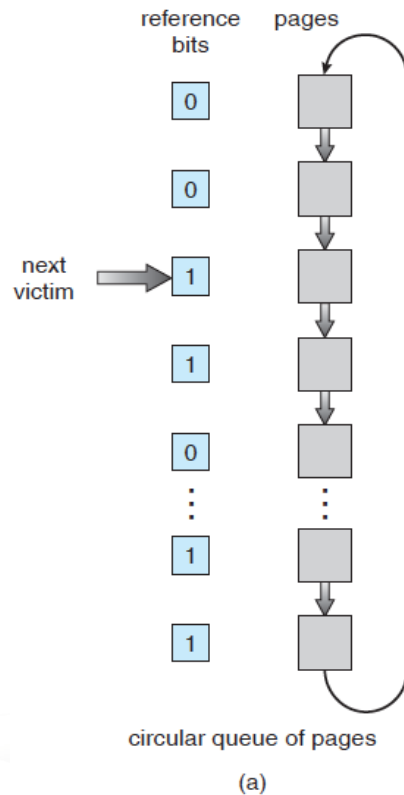
Approximate!



Approximating LRU

? Second-chance (clock) algorithm

- ? Each entry in page table has a reference bit.
- ? Whenever a page is referenced, the reference bit is set.
- ? Whenever a page is selected as a victim, we use this:
 - if reference bit == 1
 - then set reference bit to 0 and point to next as a potential victim
 - else replace page and point to the next page as a potential victim



Allocation Policies

- How many frames should be allocated to a process?
 - Minimum and maximum allocation
 - Warning: **Belady's Anomaly**
 - Static allocation
 - What is the optimal degree of multiprogramming?
 - Trashing
 - Dynamic Allocation
-
- Read Sections 8.5 and 8.6

Allocation Algorithms

? **Problem** : determine how many frames to allocate to a process

? Let M = total number of frames in the physical memory

? D_i = the number of frames allocated to Process P_i

? S_i = the number of pages in the virtual memory of P_i

? **Minimum** number of frames:

? D_i : maximum number of frames involved in an instruction. (CPU dependent)

? **Maximum** number of frames:

? $D_i \leq \min(M, S_i)$

Comparing Allocating 3 versus 4 frames

- FIFO : replace the page which comes earliest

- pages allocated to a process = 3

• pages allocated to a process = 0																																																
Page Ref.	1	2	3	4	1	2	5	1	2	3	4	5																																				
Me. State	<table><tr><td>1</td></tr><tr><td>-</td></tr><tr><td>-</td></tr></table>	1	-	-	<table><tr><td>1</td></tr><tr><td>2</td></tr><tr><td>-</td></tr></table>	1	2	-	<table><tr><td>1</td></tr><tr><td>2</td></tr><tr><td>3</td></tr></table>	1	2	3	<table><tr><td>4</td></tr><tr><td>2</td></tr><tr><td>3</td></tr></table>	4	2	3	<table><tr><td>4</td></tr><tr><td>1</td></tr><tr><td>3</td></tr></table>	4	1	3	<table><tr><td>4</td></tr><tr><td>1</td></tr><tr><td>2</td></tr></table>	4	1	2	<table><tr><td>5</td></tr><tr><td>1</td></tr><tr><td>2</td></tr></table>	5	1	2	<table><tr><td>5</td></tr><tr><td>1</td></tr><tr><td>2</td></tr></table>	5	1	2	<table><tr><td>5</td></tr><tr><td>1</td></tr><tr><td>2</td></tr></table>	5	1	2	<table><tr><td>5</td></tr><tr><td>3</td></tr><tr><td>2</td></tr></table>	5	3	2	<table><tr><td>5</td></tr><tr><td>3</td></tr><tr><td>4</td></tr></table>	5	3	4	<table><tr><td>5</td></tr><tr><td>3</td></tr><tr><td>4</td></tr></table>	5	3	4
1																																																
-																																																
-																																																
1																																																
2																																																
-																																																
1																																																
2																																																
3																																																
4																																																
2																																																
3																																																
4																																																
1																																																
3																																																
4																																																
1																																																
2																																																
5																																																
1																																																
2																																																
5																																																
1																																																
2																																																
5																																																
1																																																
2																																																
5																																																
3																																																
2																																																
5																																																
3																																																
4																																																
5																																																
3																																																
4																																																
Page Fault ?	y	y	y	y	y	y	y	n	n	y	y	n																																				

Page fault rate = 9/12

Page Replacement Algorithms

- FIFO : replace the page which comes earliest

- pages allocated to a process = 4

Page Ref.	1	2	3	4	1	2	5	1	2	3	4	5
Me. State	1 - - -	1 2 - -	1 2 3 -	1 2 3 4	1 2 3 4	1 2 3 4	5 2 3 4	5 1 3 4	5 1 2 4	5 1 2 3	4 1 2 3	4 5 2 3
Page Fault ?	y	y	y	y	n	n	y	n y	n y	y	y	n y

Page fault rate = 10/12 !!! While the number of pages increased, the p.f.r increases as well ! This is called **Belady's anomaly** !

Static and Dynamic Allocation Policies

- Static allocation
 - What is the optimal degree of multiprogramming?
 - Trashing
 - Dynamic Allocation
-
- Read Sections 8.5 and 8.6

Static Allocation Algorithm

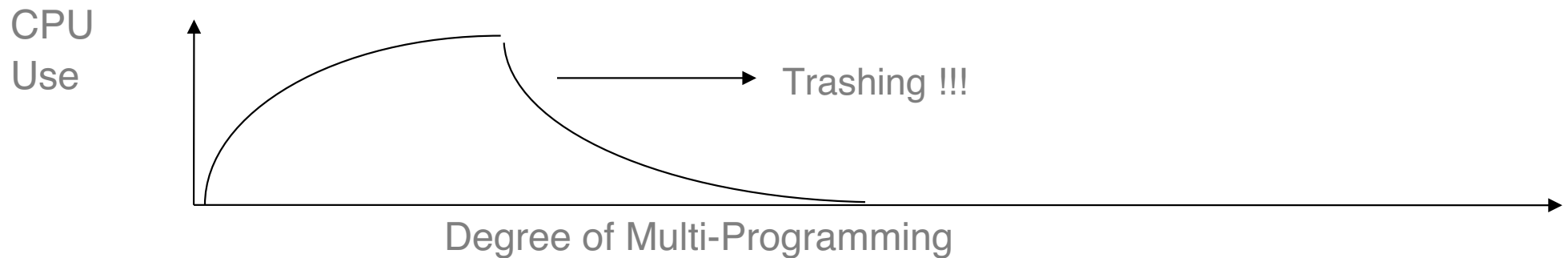
? Equal algorithm : $D_i = M/n$ where n is the number of processes.

? Proportional algorithm

? $D_i = M * S_i / S$, where $S = \sum S_i$

Determining the Degree of Multiprogramming

- ? Monitoring the CPU utilization
- ? If CPU utilization is too low, increase n (**degree of multiprogramming**)
- ? If CPU utilization is too high, reduce n .
- ? Problem ?
- ? Will increasing n always result in increasing CPU utilization ?



- ? A process is **trashing** if it is spending more time paging than executing
- ? **A system is trashing** if increasing the degree of multi-programming reduces CPU utilization.

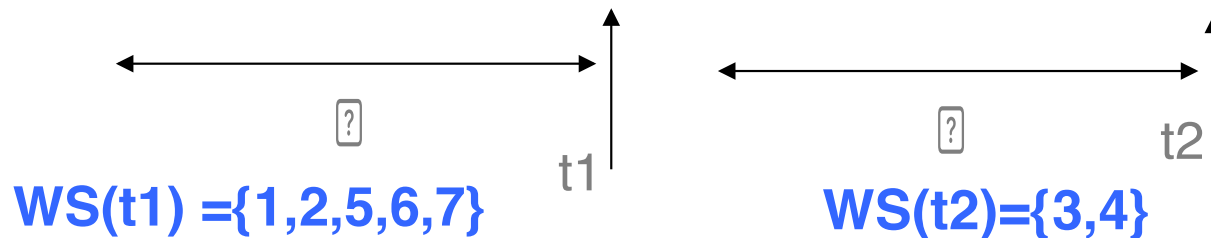
Prevention of Trashing

- ❑ Allocate sufficient number of frames to each process in memory.
- ❑ Increasing the degree of multi-programming only if the current processes have had enough frames and CPU utilization is still low.

How many frames are enough to a process ?

- ? Working set **rational**: based on temporal locality : the most recent accessed frames will be used again
- ? Working set **definition**: The working set of a process is the **set** of most recent ? page references. It varies as the process executes.

...2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3..



Dynamic Allocation Algorithm

□ $D_i = W_{si}(t,)$

□ If we can select □ properly and the process has locality behavior, then keeping W_{si} pages for the process will prevent its trashing.

Wrap Up

- Review the objectives, constraints, and “helpers” of Memory Management.
- Learn and understand the design guiding principles to hierarchical memory.
- Describe, discuss and evaluate the virtual memory (Demand Paging).
- Learn and understand Page Replacement Policies.
- Learn and Understand Frame Allocation Policies.
- Responsible for reading Sections 1.8.3, 8.1 through 8.6.